

MTech Project Ideas: ML in VLSI

<https://gemini.google.com/app/b0c5985bf105a057>

User prompt: help me think of & decide mtech 1 year project topic (especially keeping in mind data availability preferably public data). it would be helpful but not necessary if it helps HCL. I would be working on this in addition to my HCL day to day tasks. Approval i have to get from mentor Arnavesh (easier) and IIT Madras (they will essentially approve or reject based on if it's significant enough ML component or not for 1 yer mtech project) Attached background info of me and manager/mentor Arnavesh is it would be if possible better to work with something i have at least some familiarity with so that i can actually get it done. 1. first search some (especially recent) mtech thesis papers of projects done at IITs or other places 2. search / consider the current state of ML now in 2026 3. help me think of project what to do

Response: Based on your extensive history with model compression (**AutoDNN** for STM32 chips), your strong software engineering profile, your manager Arnavesh's deep background in edge AI and hardware architectures, and your current team alignment with HCL's VLSI workflows, here are three high-potential MTech project options.

All options fulfill **IIT Madras's requirement** for a rigorous, multi-approach ML component (beyond out-of-the-box usage) and utilize **publicly available datasets** to bypass HCL data clearance issues.

Option 1: Hardware-Aware Neural Architecture Search (HW-NAS) with Mixed-Precision Quantization Policy Optimization

- **The Core Idea:** Instead of manually pruning layers (like in AutoDNN), you develop an automated reinforcement learning (RL) or evolutionary algorithm-based framework that searches for the optimal neural network architecture alongside its layer-wise mixed-precision quantization policy. The search constraints are directly mapped to target microcontrollers (e.g., specific RAM/MACC limits for edge chips).
- **Why it fits your background:** It is an architectural and algorithmic evolution of your past **AutoDNN** project.
- **Why IIT Madras will approve:** It requires a significant, complex ML implementation (RL or Genetic Algorithms optimizing multi-objective functions: validation accuracy vs. hardware constraints). You can perform detailed ablation studies on search algorithms.
- **Public Datasets to Use:** ImageNet-mini, COCO, or specialized edge computer vision datasets like **PlantVillage** or **Food-101** (which you used previously).
- **HCL Synergy / Mentor Buy-in:** Extremely valuable for Arnavesh's core expertise at the intersection of AI deployment on edge devices and hardware constraints.

Option 2: Machine Learning-Driven Routing Congestion Prediction in VLSI Designs using Graph Neural Networks (GNNs)

- **The Core Idea:** Physical design and routing take major compute time in VLSI pipelines. You frame a netlist or macro placement configuration as a graph and train a **Graph Neural Network (GNN)** to predict routing congestion hotspots or total wire length early in the design loop before traditional electronic design automation (EDA) tools run routing simulations.
- **Why it fits your background:** You are currently embedded with the VLSI SoC team at HCL. While you are not a VLSI domain expert, your strong software background will allow you to focus on the graph data structures, feature encoding, and GNN model training.
- **Why IIT Madras will approve:** GNNs applied to spatial optimization represent a cutting-edge domain in machine learning. It provides plenty of ground for exploring multiple architectures (GCN vs. GAT vs. GraphSAGE) and performing error analyses.
- **Public Datasets to Use:** **ISPD 2015 / 2024 Benchmarks**, **CircuitNet** (an open-source dataset explicitly built for ML applications in VLSI physical design), or **MacroPlace** datasets.
- **HCL Synergy / Mentor Buy-in:** Directly builds an automated ML layer complementing your team's current work with HCL's Falcon hardware team. Arnavesh's hardware and design engineering expertise makes him an ideal domain mentor for this.

Option 3: Cross-Device Cross-Architectural Knowledge Distillation (KD) Framework for Heterogeneous Edge Accelerators

- **The Core Idea:** Design an optimization engine that distills the logic of huge vision or multimodal models into light, customized student models targeting completely different hardware topologies (e.g., shifting from GPU-trained structures to NXP or STM hardware-friendly execution). You evaluate multi-teacher distillation techniques vs. progressive distillation approaches.
- **Why it fits your background:** Directly utilizes your Azure GPU server access for training/distillation, and scales up your familiarity with classification and object detection models (YOLO, ResNet, MobileNet).
- **Why IIT Madras will approve:** Knowledge Distillation requires building custom loss functions (combining Kullback-Leibler divergence with task-specific losses) and comparing diverse architectural pairings.
- **Public Datasets to Use:** **COCO (person/full)** and **MobileNet baseline zoos**.

- **HCL Synergy / Mentor Buy-in:** Arnavesh has explicit experience with hardware computer vision accelerators (ADAS) and edge deployments. This tool directly aids fast deployment pipelines.

Strategy to Secure Quick Dual-Approvals

1. **For Mentor Arnavesh:** Pitch **Option 1 or Option 2**. Emphasize that the use of public data removes corporate IP liabilities. Show him that the scope will be constrained cleanly within software pipelines, relying on his hardware expertise only during the problem definition phase and final evaluation metrics.
2. **For IIT Madras (Due by July 15, 2026 - Phase I Particulars):** Ensure your proposal relies heavily on mathematical framing (e.g., formulating the task as a Multi-Objective Optimization Problem under strict constraints). Do not call it a "tool" or a "pipeline"; present it as a **"Comparative Study of Machine Learning Methodologies for [Chosen Domain]."** Detail a rigorous execution plan showcasing baseline models, an ablation study, and specific evaluation metrics.

User prompt: rate estimated difficulty for each of these considering personal backgrounds. till now subjects i studied at mtech are math fundamentals for data science, python programming, data analytics lab, machine learning theory and practice, deep learning theory and practice, mlops, multi variate data analysis, online and reinforcement learning

Response: Here is a tailored difficulty rating for each project option, structured directly around your academic background (especially **Machine Learning Theory & Practice, Deep Learning, Online and Reinforcement Learning, and MLOps**) and your strong software engineering profile.

Option 1: Hardware-Aware Neural Architecture Search (HW-NAS) with Mixed-Precision Quantization Policy Optimization

- **Estimated Difficulty: Medium**
- **Why this is highly manageable for you:**
 - You just completed *Online and Reinforcement Learning*, meaning formulating an environment, an agent, actions (layer bit-widths or pruning ratios), and a multi-objective reward function (Accuracy + MACC/RAM metrics) is fresh in your mind.
 - Your deep familiarity with CNN backbones (MobileNet, ResNet) and weights deletion from your past **AutoDNN** workflow eliminates the data engineering learning curve.
 - You have completed *MLOps*, making it significantly easier to set up structured

tracking for architectural variants and deploy automated validation loops on your Azure GPU servers.

- **IITM Approval Check:** High. The combination of an RL agent optimizing a deep search space satisfies their mandate for a heavy ML core.

Option 2: Machine Learning-Driven Routing Congestion Prediction in VLSI Designs using Graph Neural Networks (GNNs)

- **Estimated Difficulty: Medium-High to High** (Depending on the approach)
- **Why it poses a challenge:**
 - While you have strong *Deep Learning* fundamentals, standard MTech courses rarely cover Geometric Deep Learning or Graph Neural Networks (GNNs) in depth. You would need to learn GNN frameworks (like DGL or PyTorch Geometric) from scratch.
 - Extracting, parsing, and transforming netlists or layouts into graph structures (Nodes as cells/macros, Edges as interconnects) is heavy data engineering.
- **How to make it a "Medium" difficulty:**
 - Instead of a GNN, frame the open-source **CircuitNet** data as an **Image-to-Image translation task**. You can treat layout regions as spatial grids (images) and train a **Fully Convolutional Network (FCN) or U-Net** to predict a congestion heatmap. This directly leverages your *Deep Learning Theory and Practice* coursework without needing to build complex graph-parsing software pipelines.
- **IITM Approval Check:** Very High. Predictive cross-stage modeling in VLSI CAD is an elite research area.

Option 3: Cross-Device Cross-Architectural Knowledge Distillation (KD) Framework for Heterogeneous Edge Accelerators

- **Estimated Difficulty: Low-Medium**
- **Why this is the easiest option to execute:**
 - It directly builds upon *Deep Learning Theory and Practice* and *MLOps*. Knowledge Distillation utilizes established optimization mechanics—specifically modifying the model's loss function to minimize the KL divergence between the soft targets of a teacher model and a student model.
 - It bypasses complex reinforcement learning loops (Option 1) and complex domain representations (Option 2). It leans heavily on your strongest asset:

writing clean, modular Python code to coordinate experiments, manage model states, and log evaluations.

- **The "IITM Trap" to watch out for:** Because the baseline implementation of KD is straightforward, the examiners might flag it as "just running state-of-the-art models" if you aren't careful. To guarantee approval, you must explore multiple structured techniques (e.g., comparing *Response-Based* vs. *Feature-Based* distillation across diverse target edge topologies) and provide a rigorous error and ablation study.

Final Recommendation

If you want the **safest, fastest path to completion** while working your day-to-day tasks at HCL, choose **Option 3**, but intentionally design a complex experimental matrix to please IITM.

If you want the **absolute best alignment with your current HCL team and mentor Arnavesh's background**, choose **Option 2** but explicitly implement it as a **U-Net/FCN Image-to-Image translation task using CircuitNet** to save yourself from complex GNN graph-engineering bottlenecks.

User prompt: Also search data availability for each

Response: Here is the concrete data availability mapping for each option. Every option relies completely on public datasets, removing any need for internal HCL data clearance.

Option 1: Hardware-Aware Neural Architecture Search (HW-NAS)

- **Data Availability: Abundant / Plug-and-Play**
- **Primary Public Datasets:**
 - **ImageNet-Mini or Tiny-ImageNet:** Standard standard benchmarks for image classification NAS loops.
 - **PlantVillage & Food-101:** You explicitly have past experience using these datasets for model pruning, so your data loading pipelines are already written.
 - **NAS-Bench-201 / HW-NAS-Bench:** These are public, pre-computed benchmark datasets that map specific neural architectures to actual hardware metrics (latency, energy) across devices like edge GPUs and microcontrollers.
- **Data Engineering Load: Low.** You do not need to collect new data; you can plug standard PyTorch/TensorFlow data loaders directly into the search loop.

Option 2: Machine Learning-Driven Routing Congestion Prediction (VLSI)

- **Data Availability: Excellent via specialized EDA open-source repos**
- **Primary Public Datasets:**
 - **CircuitNet (CircuitNet-N14 / CircuitNet-N28 / CircuitNet-N45):** This is a massive, widely recognized public dataset specifically designed for ML-in-EDA applications. It contains over 10,000 layout samples derived from open-source RISC-V designs.
 - **ISPD 2015 / 2024 Placement Benchmarks:** Standard competition netlists used to validate routing and placement congestion.
- **Why it simplifies your execution:** CircuitNet provides features preprocessed and transformed directly into **Numpy arrays**. If you choose the image-to-image translation path (U-Net/FCN), the data is already structured as two-dimensional spatial grids (representing cell density, macro regions, and congestion maps), bypassing complex graph engineering entirely.

Option 3: Cross-Device Cross-Architectural Knowledge Distillation (KD)

- **Data Availability: Universal / Maximum Availability**
 - **Primary Public Datasets:**
 - **MS-COCO (Full or Person subset):** The global standard for object detection (YOLO, SSD) distillation setups. You have historical experience working with both the full and person-specific slices of COCO.
 - **Pascal VOC:** A lighter object detection dataset that speeds up iteration times during the trimester stages if your Azure GPU resource bandwidth is tight.
 - **Data Engineering Load: Zero.** Since you are dealing with standard object detection formats, you can pull pre-written dataloaders directly from open-source vision libraries.
-

User prompt: finally search and check multiple times on google scholar, consensus and other sources whether these projects are already done meaning i can't do

Response: A literature scan of top AI conferences and Google Scholar profiles confirms that **none of these projects are "already done" in a way that prevents you from doing them.**

In academia and at IIT Madras, a project is considered "done" if you are blindly replicating a specific, existing paper's architecture without custom changes. However, exploring a known *domain* using your own comparative techniques, framework variations, or customized metrics is completely valid and highly expected for an MTech thesis.

The current research landscape for these topics indicates where the existing work stands and how you can frame your specific approach to guarantee IITM approval:

Option 1: Hardware-Aware Neural Architecture Search (HW-NAS)

- **What already exists:** State-of-the-art frameworks like *JAQ* or *LEMUR 2* explore hardware-software co-design and supernet optimization, but they focus heavily on high-end edge GPUs or specialized custom ASICs.
- **Your Novel Edge for IITM:** Most public NAS frameworks do not target mixed-precision constraints mapped specifically to lower-tier microcontrollers (like **STM32** or **NXP chips**). You can build a reinforcement learning loop explicitly optimizing structural layer pruning and mixed-precision constraints (like 2-bit, 4-bit, and 8-bit switches), using metrics directly relevant to your past work.
- **IITM Framing:** *"A Comparative Evaluation of Reinforcement Learning Policies for Mixed-Precision Quantization under Microcontroller MACC Constraints."*

Option 2: VLSI Routing Congestion Prediction (CircuitNet)

- **What already exists:** Academic papers have established that translating layout properties (RUDY, cell density) into congestion maps works well using classic U-Net or FCN architectures.
- **Your Novel Edge for IITM:** While basic image-to-image translations are mapped out, the literature shows a clear gap in extensive cross-architecture comparative ablation studies. You can implement a framework that compares standard **U-Net** against **Inception-boosted FCNs** or **Attention U-Nets** explicitly across different sub-designs of the open-source RISC-V configurations within CircuitNet.
- **IITM Framing:** *"Evaluating Spatial Attention and Cross-Stage Feature Fusion in Fully Convolutional Architectures for VLSI Routing Congestion Prediction."*

Option 3: Knowledge Distillation (KD) for Heterogeneous Edge Detectors

- **What already exists:** Advanced papers like *UniKD* or *HEAD* deal with the complex math of transferring knowledge between completely different structural formats (like distilling an anchor-based Transformer detector down into a lightweight anchor-free CNN student model).
- **Your Novel Edge for IITM:** These top-tier research papers focus on complex, custom-built transformer layers (e.g., query-based extraction) that are computationally heavy and highly volatile to train from scratch. You can focus your thesis on a **systematic structural comparison of response-based vs. feature-based distillation** exclusively using edge-ready models (e.g., distilling an Azure-hosted heavy YOLOv10 teacher model down to optimized, light MobileNet/YOLOX

variants suitable for embedded deployment).

- **IITM Framing:** *"An Ablation Study on Feature Geometry Alignment and Gradient Conflicts in Heterogeneous Object Detection Distillation for Edge Deployment."*

The Verdict

You have a clear path forward. IIT Madras will reject a proposal only if it says *"I will download CircuitNet and run a pre-made U-Net code repository."* They will enthusiastically **approve** if you say *"I am using CircuitNet to rigorously evaluate and compare three distinct encoder-decoder segmentation variants, measuring structural similarity (SSIM) performance limits under varying placement densities."*

Select the option that matches your bandwidth, set up a comparative matrix of multiple approaches, and you will easily pass the Phase I submission due on **July 15, 2026**.
